

WEEK 07 · FRONTEND 2027

THE LANGUAGE YOU ALREADY KNOW

JavaScript for people who already program — and the method you will use for the rest of the course.

WHERE WE ARE

Six weeks of pages that render and work. Today they start to think.

AFTER TODAY YOU CAN

Read JavaScript that surprises you, and say why it did.

WHAT THIS UNLOCKS

Your three pages, and the first spec you write before the code.

“Java and JavaScript are as similar as car and carpet.”

LESSON 8, KEPT — AND IT IS STILL THE MOST USEFUL SENTENCE IN THIS DECK

SIX LINES. EVERY ONE OF THEM IS LEGAL.

```
1 [] + []           // ""
2 [] + {}           // "[object Object]"
3 0.1 + 0.2          // 0.30000000000000004
4 [10, 9, 1].sort() // [1, 10, 9]
5 typeof null        // "object"
6 '5' - 3             // 2      ... but '5' + 3 is "53"
```

You will not need **any** of these lines. You will need to not be surprised by them.

TEN DAYS, IN 1995

BRENDAN EICH, NETSCAPE

Ten days to build a scripting language for the browser. It was called **LiveScript**.

THEN MARKETING HAPPENED

Renamed **JavaScript** to ride Java's popularity. The two languages are unrelated.

WHY YOU CARE

The odd parts are 1995 decisions that could never be removed. **The web does not break old pages.**

Every strange rule today is a promise kept to a page written in 1997.

WHERE YOUR CODE RUNS NOW

What it is

One thread, inside a tab, next to a document it can read and change.

Key trait

There is no `main`. Your code runs, finishes, and the page keeps living. Everything after that is a reaction.

Use cases

This week: the language only. Next week it meets the page.

Tips

The console is your REPL. `console.log`, never `alert`.

THE ONE THAT MATTERS

One thread. While your code runs, nothing else can — not a click, not a timer, not the page repainting. Act 7 is entirely about the consequences.

WHAT TODAY IS, AND IS NOT

NOT TODAY

Variables. Loops. `if`. Functions as an idea. Arrays as an idea. You have all of it.

TODAY

The seven places where what you know is **wrong here**, and the idioms your project is built from.

SECOND HALF

The method: chat to think, a spec to decide, Claude Code to write, and a review that is graded.

► **SAY THE DEAL OUT LOUD** — if any of this is genuinely new rather than merely different, `primer/` covers it, with solutions, and nobody is graded on it.

SEVEN TYPES, AND ONE OF THEM IS A MISTAKE

PRIMITIVES

`string` · `number` · `boolean` ·
`undefined` · `null` · `symbol` ·
`bigint`

EVERYTHING ELSE

`object` — and arrays, functions
and dates are all objects wearing
hats.

THE MISTAKE

`typeof null === "object"`. A
1995 bug, never fixable. Test with
`value === null`.

```
1 let count = 5;           // number
2 count = 'five';          // still legal. the TYPE lives in the value, not the variable
```

THE TABLE NOBODY MEMORISES, COMPUTED LIVE

every cell is a real `a == b`, and it explains itself

↺ reload

open ↗

LOOSE EQUALITY, EVALUATED BY THIS PAGE

☒ compare with == ☐ compare with === ☐ highlight the broken triangle

Click a cell to see how the specification got there.

==	0	1	'0'	''	'false'	false	true	null	undefined	NaN	[]	[0]
0	true	false	true	true	false	true	false	false	false	false	true	true
1	false	true	false	false	false	false	true	false	false	false	false	false
'0'	true	false	true	false	false	true	false	false	false	false	false	true
''	true	false	false	true	false	true	false	false	false	false	true	false
'false'	false	false	false	false	true	false	false	false	false	false	false	false
false	true	false	true	true	false	true	false	false	false	false	true	true
true	false	true	false	false	false	false	true	false	false	false	false	false
null	false	false	false	false	false	false	false	true	true	false	false	false
undefined	false	false	false	false	false	false	false	true	true	false	false	false
NaN	false	false	false	false	false	false	false	false	false	false	false	false
[]	true	false	false	true	false	true	false	false	false	false	false	false
[0]	true	false	true	false	false	true	false	false	false	false	false	false

35 of 144 pairs are equal under ==. The diagonal accounts for 9 of them. Everything else is a conversion you did not ask for.

HOW IT GOT THERE

Pick a cell.

► ASK FOR `'' == 0`, THEN `0 == '0'`, THEN `'' == '0'` — and wait. Only then switch on the broken triangle.

THE RULE, AND THE ONE EXCEPTION WORTH KEEPING

NEVER

```
1 if (value == 0)
2 if (input == false)
3 // "" passes. [] passes.
```

ALWAYS

```
1 if (value === 0)
2 if (input === false)
```

THE EXCEPTION

```
1 if (value == null)
2 // null AND undefined,
3 // and nothing else
```

=== asks one question. **==** asks a question and then negotiates.

EIGHT VALUES ARE FALSE. EVERYTHING ELSE IS NOT.

sixteen values, sorted by an actual `if`

reload

open

TWO DIFFERENT QUESTIONS ABOUT THE SAME SIXTEEN VALUES

☒ mark the three that arrive from a form or from JSON

`if (value)` – "is it truthy?"

FALSY

false

0

-0

0n

''

null

undefined

NaN

8 of 16

TRUTHY

true

1

-1

'0'

'false'

''

[]

{}

8 of 16

`value == false` – "is it equal to false?"

EQUAL TO FALSE

false

0

-0

0n

''

'0'

''

[]

8 of 16

NOT EQUAL TO FALSE

null

undefined

NaN

true

1

-1

'false'

{}

8 of 16

6 values land in opposite bins in the two columns: null · undefined · NaN · '0' · '' · []. "Falsy" and "equal to false" are not the same question, and every value above is a value you can actually be holding.

Falsy is not a value and it is not a comparison. It is what the language does when it needs a yes or a no and you handed it something else — in an `if`, in `&&`, in `||`, in a ternary. That is a different question from `== false`, and the two columns above disagree about real values you will actually be holding.

► COVER THE SCREEN AND ASK FOR A NUMBER FIRST. Then look at `[]` — truthy in one column, equal to `false` in the other.

TWO WAYS TO HAVE NOTHING, AND ONE TO HAVE A WRONG NUMBER

undefined

The language's absence. A variable never assigned, a missing property, a function with no `return`.

null

Your absence. You wrote it, on purpose, to mean "known to be empty".

NaN

A `number`. It is not equal to itself. `Number.isNaN(x)` is the test.

WHERE NAN COMES FROM

```
1 Number('12 kg')    // NaN
2 undefined + 1       // NaN
3 [10,9].sort()[0] - 1 // fine, but...
```

NaN never throws. It travels quietly through arithmetic and appears on screen four functions later.

CONVERTING ON PURPOSE

```
1 Number('42')      // 42
2 Number('')         // 0      ← the empty string is zero. remember this in week 10.
3 Number('42kg')     // NaN
4
5 String(42)         // "42"
6 Boolean('')        // false
7 Boolean('false')   // true    ← a non-empty string. all of them.
8
9 Number.parseInt('42kg', 10) // 42  - stops at the first non-digit
10 Number.parseFloat('3.5rem') // 3.5
```

When a form gives you
"7" and you compare it to 7,
convert — do not let == guess.

YOU KNOW THIS ONE. IT IS NOT THE SAME HERE.

IN C#

A `struct` is copied. A `class` is shared. **You decide, when you write the type.**

IN JAVASCRIPT

The seven primitives are copied. **Everything else is shared.** You do not decide, and there is no keyword.

SO

Every array, every object, every function you pass to anything is the **original.**

There is no `struct`. There is no `ref`. There is one rule, and it is not written anywhere in your code.

SIX PROGRAMS, AND THE ENGINE ANSWERS

identical steps, one difference – and an `a === b` asked after each

⌕ reload open ↗

SIX PROGRAMS, STEPPED, WITH THE ENGINE ANSWERING

Program 1 · a number is copied ▼ Step Reset step 0 of 3

```
let a = 10;  
let b = a;  
b = 99;
```

Press Step.

BINDINGS, AFTER THIS STEP

NAME	VALUE
a	undefined
b	undefined

► **STOP PROGRAM 4 ON ITS LAST STEP.** Ask who thinks `prices` is unchanged. Count the hands. Then press Step.

THE FUNCTION THAT TOLD THE TRUTH AND LIED

WHAT IT SAYS

```
1 function addTax(list) {  
2   list.sort((a, b) => a - b);  
3   return list.map((p) => p * 1.17);  
4 }
```

Returns a new array. **And reorders yours.**

WHAT IT SHOULD SAY

```
1 function addTax(list) {  
2   const copy = [...list];  
3   copy.sort((a, b) => a - b);  
4   return copy.map((p) => p * 1.17);  
5 }
```

One line. Now the name is honest.

sort and **reverse** mutate **and**
return the array, so they look safe.

const IS ABOUT THE NAME

```
1  const list = [1, 2];
2  list.push(3);           // fine. the array was never frozen.
3
4  const total = 0;
5  total = 1;              // TypeError: Assignment to constant variable.
6
7  // use const everywhere. use let when the NAME must point somewhere else.
```

THE HABIT

`const` by default. `let` when you reassign. `var` never — act 4 says why.

NOT A PROMISE OF IMMUTABILITY

If you actually need that, copy: `[...list]`, `{ ...item }`. Both are shallow.

THREE WAYS TO DECLARE, AND THEY FAIL DIFFERENTLY

the same three lines – and this page throws a real error while it loads

reload

open

THE SAME THREE LINES, THREE DECLARATIONS

WHAT THIS PAGE DID TO ITSELF WHILE LOADING

The browser threw this, uncaught, before you clicked anything. It is in your console too, in red: `Uncaught ReferenceError: Cannot access 'answer' before initialization`

var

let

const

function

typeof, both kinds

```
console.log(x);  
var x = 5;  
console.log(x);
```

WHAT ACTUALLY HAPPENED

line 1 undefined

line 3 5

No error. `var x` was hoisted to the top of the function and initialised to `undefined`, so line 1 is legal and silent. The mistake is still a mistake — it just gets reported somewhere else, later, usually as NaN.

► **OPEN DEVTOOLS BEFORE THIS ONE.** Run `var` first and wait for somebody to say it is safer. Then run `let`.

HOISTED IS NOT THE SAME AS USABLE

`var`

Hoisted to the top of the **function**, initialised to `undefined`. Reading early is legal and silent.

`let` / `const`

Hoisted to the top of the **block**, **not initialised**. Reading early throws. That gap is the temporal dead zone.

function declaration Hoisted with its body. Calling it above its definition works.

THE ONE THAT COSTS MONEY

`var` does not fail. It produces `undefined`, which becomes `NaN`, which reaches the screen. **The error message is a symptom four functions from the cause.**

A CLOSURE IS A FUNCTION THAT REMEMBERS WHERE IT WAS BORN

the puzzle, the fix, and then the version you are graded on

↺ reload

open ↗

THE LOOP EVERYBODY HAS SEEN, AND THE PATTERN YOU ARE ABOUT TO SHIP

1 — THREE FUNCTIONS, ONE VARIABLE

```
for (var i = 0; i < 3; i++) {  
  jobs.push(() => i);  
}  
jobs.map((j) => j());
```

Run

The functions are fine. They all close over the same `i`, and by the time any of them runs, the loop has finished and that one `i` is 3.

2 — THREE FUNCTIONS, THREE VARIABLES

```
for (let i = 0; i < 3; i++) {  
  jobs.push(() => i);  
}  
jobs.map((j) => j());
```

Run

One keyword. `let` creates a fresh binding per iteration, so each function closes over a different one. Nothing about the functions changed.

3 — THE REASON ANYONE CARES

```
function createStore() {  
  let state = { count: 0 };  
  const listeners = new Set();  
  return {  
    getState: () => state,  
    setState(patch) { ... },  
    subscribe(fn) { ... },  
  };  
}
```

setState count + 1

subscribe

unsubscribe

try to reach in

`state` is a plain `let` inside a function that has already returned. Nothing on this page can name it. That is the same closure as panel 1 — and here it is the reason your state cannot be corrupted from outside its file.

► **AFTER PANEL 2, SAY IT OUT LOUD:** "that was the puzzle version — here is the one your project is built out of." Then panel 3.

THE SAME CLOSURE, IN THE FILE YOU WILL SHIP

```
1 let state = initialState(); // nothing outside this file can name it
2 const listeners = new Set();
3
4 export const getState = () => state;
5 export function setState(patch) {
6   state = { ...state, ...patch };
7   for (const listener of listeners) listener(state);
8 }
9
10 export function subscribe(fn) { listeners.add(fn); return () => listeners.delete(fn); }
```

This is `starter-repo/js/state.js`.
You already have it. In week 9 you fill it in.

LIVE: BUILD THE STORE, IN NINE LINES

TOGETHER, IN THE CONSOLE

A counter with private state. `getState`, `setState`, `subscribe`. Then subscribe `console.log` and press it.

THEN BREAK IT ON PURPOSE

Move `let state` outside the function. Everything still works — and now anyone can write to it.

► **LIVE-CODING BREAK 1 OF 2.** Nine lines, typed, not pasted. Roughly six minutes.

FUNCTIONS ARE VALUES. THAT IS NOT A SLOGAN.

```
1 const jobs = [shout, trim, translate]; // stored, like any value
2
3 subscribe(render); // passed, WITHOUT calling it
4
5 function makeComparator(key) { // returned, built to order
6   return (a, b) => a[key].localeCompare(b[key], 'he');
7 }
```

`subscribe(render)` hands over the function. `subscribe(render())` calls it and hands over its **result**.

ARROW OR FUNCTION – AND IT IS NOT ABOUT BREVITY

SAME

```
1 const a = (x) => x * 2;  
2 function b(x) { return x * 2; }
```

THE TRAP

```
1 const f = (x) => { x * 2 };  
2 f(3); // undefined
```

Braces make a **body**. A body needs `return`.

EITHER OF THESE

```
1 const f = (x) => x * 2;  
2 const g = (x) => {  
3   return x * 2;  
4 };
```

Returning an object needs the parentheses:

`(item) => ({ id: item.id })`.

Without them the braces are a body again.

ONE FUNCTION, FIVE CALL SITES

nothing is redefined between the rows – only the call changes

↺ reload

open ↗

ONE FUNCTION OBJECT, FIVE CALL SITES

```
const owner = { name: 'the shelf' };

function probe() {
  return this?.name ?? '(no this)';
}

owner.probe = probe;           // the SAME function, given a second name
```

Run all five

☐ define `probe` as an arrow instead

HOW IT IS CALLED	THIS?.NAME	WHY
<code>probe()</code>	(no this)	Nothing before the dot, because there is no dot. In a module (and in strict mode) <code>'this'</code> is <code>'undefined'</code> rather than the window.
<code>owner.probe()</code>	the shelf	The object before the dot becomes <code>'this'</code> . This is the rule people think is the only rule.
<code>probe.call(other)</code>	a different object	<code>'call'</code> , <code>'apply'</code> and <code>'bind'</code> say what <code>'this'</code> is, out loud. This is the escape hatch.
<code>[1].map(owner.probe)[0]</code>	(no this)	The function was passed, not called through the object. <code>'map'</code> calls it with no object in front, so the object is gone. This is the bug you will hit in week 8.
<code>[1].map(() => owner.probe())[0]</code>	the shelf	The arrow is the callback; the call inside it is still a method call through <code>'owner'</code> . The fix is not "use an arrow" — it is "keep the dot".

`probe === owner.probe` is true. There is one function. It does not belong to `'owner'`, it is not a copy, and nothing about it changed between the rows above — only the call site did.

► **READ THE FIVE CALL SITES ALOUD AND VOTE ON EACH, BEFORE REVEALING.** The method row is unanimous. The callback row is not.

FOUR RULES, CHECKED IN THIS ORDER

- 1 new**
a brand-new object. Not used in this course.
- 2 call / apply / bind**
whatever you said, explicitly.
- 3 a dot**
`owner.probe()` — the thing before the dot.
- 4 none of the above**
`undefined` in a module. Not the window.

AND OUTSIDE ALL FOUR

An **arrow** has no `this` of its own, so none of the rules apply. It uses the `this` of wherever it was written.

THE PRACTICAL RULE

Arrow for callbacks. Function for methods. That is the whole decision, and it is what the demo just showed.

SIX METHODS, AND YOU ALREADY KNOW FIVE OF THEM

```
1 items.map((i) => i.title)           // same length, transformed
2 items.filter((i) => i.year > 2000)   // fewer, same items
3 items.find((i) => i.id === wanted)   // one item, or undefined
4 items.some((i) => i.rating === 0)    // a boolean
5 items.every((i) => i.title)          // a boolean
6 items.reduce((sum, i) => sum + i.price, 0) // one value, any shape
```

ALL SIX

Return something new. **None of them touches the original.**

THE SEVENTH

`sort` is not like them. Next slide, and it is the misconception of the act.

THE ONE THAT IS NOT LIKE THE OTHERS

three ways to sort, and what each one actually compared

↻ reload

open ↗

THREE WAYS TO SORT THE SAME LIST

Data numbers ☐ sort a copy: [...list].sort(...) ☒ show the comparisons

Numbers. Bare sort() and localeCompare both order these as text, so both are wrong here and only the subtraction comparator is right.

list.sort()

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 10, 25, 3, 9]

ORIGINAL AFTERWARDS [1, 10, 25, 3, 9] ← reordered

WHAT WAS COMPARED

1. "10" vs "9" → first
2. "9" vs "1" → second
3. "1" vs "25" → first
4. "25" vs "3" → first

No comparator. Everything becomes a string first.

list.sort((a, b) => a - b)

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 3, 9, 10, 25]

ORIGINAL AFTERWARDS [1, 3, 9, 10, 25] ← reordered

WHAT WAS COMPARED

1. 9 vs 10 → first
2. 1 vs 10 → first
3. 1 vs 9 → first
4. 25 vs 9 → second
5. 25 vs 10 → second
5. 3 vs 10 → first
7. 3 vs 9 → first

Numbers only. Subtraction gives the negative / zero / positive that sort wants.

list.sort((a, b) => a.localeCompare(b, 'he'))

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 10, 25, 3, 9]

ORIGINAL AFTERWARDS [1, 10, 25, 3, 9] ← reordered

WHAT WAS COMPARED

1. 9 vs 10 → second
2. 1 vs 9 → first
3. 1 vs 10 → first
4. 25 vs 10 → second
5. 25 vs 9 → first
5. 3 vs 25 → second
7. 3 vs 9 → first

Text. Knows that 'a' comes before 'B'; and knows the Hebrew alphabet.

sort with no argument converts every element to a string and compares those strings, by code unit. That is not a bug and it has never changed — it is simply not the question you were asking. And it sorts the array you handed it, in place.

► **RUN HEBREW ONLY BEFORE HEBREW + ENGLISH**. The first agrees with everything and teaches the wrong lesson. That is the point.

COMPARATORS, AND THE ONLY TWO YOU NEED

WRONG

```
1 list.sort()
2 list.sort((a, b) => a.title > b.title)
```

The first compares strings. The second returns `true / false`, and `sort` wants a **number**.

RIGHT

```
1 [...list].sort((a, b) => a.year - b.year)
2 [...list].sort((a, b) =>
3   a.title.localeCompare(b.title, 'he'))
```

Negative, zero, positive. And a copy, so the caller keeps its order.

Ties are not free: `a.rating - b.rating || a.title.localeCompare(b.title, 'he')`.

TAKING THINGS APART, AND PUTTING THEM TOGETHER

```
1  const { title, year = 'unknown' } = album;    // destructuring, with a default
2  const [first, ...rest] = items;               // and rest
3
4  const next = { ...state, query: 'kid a' };     // spread: copy, then override
5  const grown = [...items, newItem];            // never push into state
6
7  function search(term, { retries = 1, timeoutMs = 8000 } = {}) { ... }
8
9  const label = `${album.title} - ${album.year}`; // template literal
```

ORDER MATTERS

`{ ...state, query }` and `{ query, ...state }` are different. The **last** one wins.

BOTH ARE SHALLOW

A nested object inside the copy is still the **same** object. Act 3, again.

TWO OPERATORS THAT REPLACE TEN LINES OF GUARDING

```
1 payload?.results?.[0]?.title      // undefined instead of a TypeError
2 album.rate?.()                     // only if it is actually a function
3
4 const year = album.year ?? 'unknown'; // only null and undefined
5 const year = album.year || 'unknown';  // ALSO 0, '' and NaN ← the bug
6
7 items.find((i) => i.id === id) ?? null // find returns undefined; be explicit
```

A rating of `0` is a rating.

`||` throws it away; `??` keeps it.

MODULES: EVERY FILE HAS A DOOR

EXPORTING — A DOOR OUT OF THE FILE

```
1 export const getState = () => state;  
2 export function selectVisible(state) { ... }
```

IMPORTING — AND THE `.js` IS REQUIRED

```
1 import { selectVisible } from './state.js';  
2 // it is a URL, not a module name
```

TWO THINGS THAT BITE

Modules are always **strict**. And `type="module"` is **deferred** — the DOM already exists when it runs, so there is no `DOMContentLoaded` anywhere in your project.

```
1 <script type="module" src="js/app.js"></script>  <!-- and it will not work over file:// -->
```

WHAT DOES THIS PRINT?

```
1 console.log('A');  
2 setTimeout(() => console.log('B'), 0);  
3 Promise.resolve().then(() => console.log('C'));  
4 console.log('D');
```

Take a vote. Write the four letters on the board before anybody runs it.

ONE STACK, TWO QUEUES, ONE RULE

the model steps – then the browser runs it, and the two are compared

ONE STACK, TWO QUEUES, AND ONE RULE

Program 1 · the classic

Step Reset Predict the output Run it for real step 0 of 7

```
console.log('A');

setTimeout(() => {
  console.log('B');
}, 0);

Promise.resolve().then(() => {
  console.log('C');
});

console.log('D');
```

CALL STACK

MICROTASKS

TASKS

THE MODEL PREDICTS

THE BROWSER ACTUALLY DID

Press Step, or Predict the output to skip to the end.

► **PROGRAM 1, THEN PROGRAM 2.** In program 2 the timer is registered *first* and still runs last. That is the whole mechanism.

THE RULE, IN ONE SENTENCE

- 1 Run**
the synchronous code, to the end. Nothing interrupts it.
- 2 Drain**
every microtask — promises, `await` — including ones added while draining.
- 3 Take one**
task: a timer, a click, a network response.
- 4 Repeat**
from step 2. Microtasks again, before the next task.

**The queues are only consulted
when the stack is empty.
Everything else follows from that.**

`setTimeout(fn, 0)` MEANS "WHEN I AM FREE"

WHAT PEOPLE READ

"Run this in zero milliseconds." It **never means that.**

WHAT IT MEANS

"Put this in the task queue. Run it after the stack empties and the microtasks are done."

MEASURED, IN DEMO 8

A synchronous loop after the timer
⇒ the callback waited **~250 ms** for its zero. Your own code decided that.

One thread. If you are busy, nothing else happens — not a click, not a repaint.

FROM TODAY, CLAUDE WRITES CODE FOR YOU

WEEKS 1-4 – TUTOR

It explained. You typed every character.

WEEKS 5-6 – ASSISTANT

It produced snippets. You justified every line you kept.

WEEKS 7-12 – PARTNER

It implements. **You decide what, and you check what came back.**

**What is graded is not whether you used it.
It is whether you know what is in your code.**

THE LOOP

Claude chat

- 1 to think. Ask for options.

No code yet.

- 2 **FEATURE_SPEC.md**

to decide. Written down is checkable.

- 3 **Claude Code**

to implement — from the spec, not from a sentence.

- 4 **Review**

against your own criteria. `git diff`, not memory.

- 5 **Iterate the spec**

, not the output. **Nobody does step 5.** It is the one that compounds.



THE SAME FEATURE, FROM TWO REQUESTS

both implementations are real, and both are run against the same eight cases

↺ reload

open ↗

ONE FEATURE, TWO REQUESTS, EIGHT CASES

☐ only run the happy path Click a row to see which line of the spec it comes from.

THE REQUEST MOST PEOPLE SEND

"Add a search box that filters the list."

1. — nothing else was said —

```
export function filterItems(items, query) {  
  return items.filter((item) =>  
    item.title.includes(query)  
  );  
}
```

fails 4 of 8

THE SAME FEATURE, SPECIFIED

"Implement `filterItems` per the spec below."

1. Section 2 — `query` is a string on state; it starts as `''`.
2. Section 5 — an empty query shows everything, not nothing.
3. Section 5 — leading and trailing spaces are trimmed before matching.
4. Section 5 — matching is case-insensitive.
5. Section 5 — an item whose title is missing must not crash the list.
6. Section 6 — with no matches the list is empty and the counter reads 0.

```
export function filterItems(items, query) {  
  const term = query.trim().toLowerCase();  
  if (term === '') return items;  
  return items.filter((item) =>  
    (item.title ?? '').toLowerCase().includes(term)  
  );  
}
```

passes all 8

► **TICK "ONLY RUN THE HAPPY PATH" FIRST.** Both green, both finished-looking. Then untick it, and say nothing.

WHAT A SPEC HAS THAT A SENTENCE DOES NOT

- §1 What the user can do after, and could not before. One sentence. If it contains "and", it is two features.
- §2 What changes in `state` — before and after.
This week that is your data model.
- §5 Edge cases, at least three. **AI writes the happy path very well and the edges almost never.**
- §6 How you know it works — things you can **see**, not "the function returns an array".
- §7 The review, written **after** the code exists.
This is the graded part.

YOU ALREADY HAVE THE FILE

`templates/FEATURE_SPEC.md` . Copy it once per feature into `specs/` .

THIS WEEK ONLY

§4 (the events) is written "**אין עדיין — שבוע 8**".
There is no behaviour yet, and pretending otherwise is worse than an empty section.

A SPEC YOU CAN CHECK BEFORE YOU READ ANY CODE

NOT A SPEC

"Filter the list." · "Handle errors properly." · "Make it responsive." · "Show a suitable message."

A SPEC

"An empty query shows everything." · "Matching ignores case." · "With no matches the list is empty and the counter reads 0."

1

Testable

could somebody else tell whether it happened, without asking you?

2

Bounded

one feature. One file. No "and".

3

Edged

empty, missing, duplicated, very long, none.

4

Worded

the actual text that appears on screen, not "a suitable message".

LIVE: ONE WINDOW, ONE FEATURE, ONE COMMIT

1 Left

Claude in the browser — three sentences about the feature, no code requested.

2 Middle

`specs/filter-and-sort.md` open, sections 1, 2, 5 and 6 filled.

3 Terminal

Claude Code, pointed at the spec.

4 Source Control

read the diff. Find the thing nobody asked for.

5 Commit

and write \$7 — what came back, what was wrong, what I did.

► **LIVE-CODING BREAK 2 OF 2.** Do it in one VS Code window, on the projector, roughly eight minutes. **NO SUBSCRIPTION?** Paste the spec into free Claude in the browser and copy the files back — the review step is identical, and it is the step worth points.

THREE QUESTIONS

- 1 Why is `'' == '0'` false
when `'' == 0` and `0 == '0'` are both true?
- 2 What does `[10, 9, 1].sort()` return
and what did it do to the array?
- 3 Why does `setTimeout(fn, 0)` run last
even when it was registered first?

If you can answer these three, you can read the week-8 code. If not, the demos are in `demos/` and they are yours.

THIS WEEK

CA7 – IN THE LAB, NOW

Predict the output. Implement four pure functions. Review one function Claude wrote. Write your first spec, and build the three pages.

THE MILESTONE

Three pages in HTML and Tailwind, with invented data in the markup, and navigation between them. **No JavaScript in the project yet.**

BEFORE NEXT WEEK

Nothing to install. If any of today felt new rather than merely different, `primer/` is there, with solutions.

► **SUBMISSION:** Moodle, repo URL + commit SHA, before week 8 begins.